

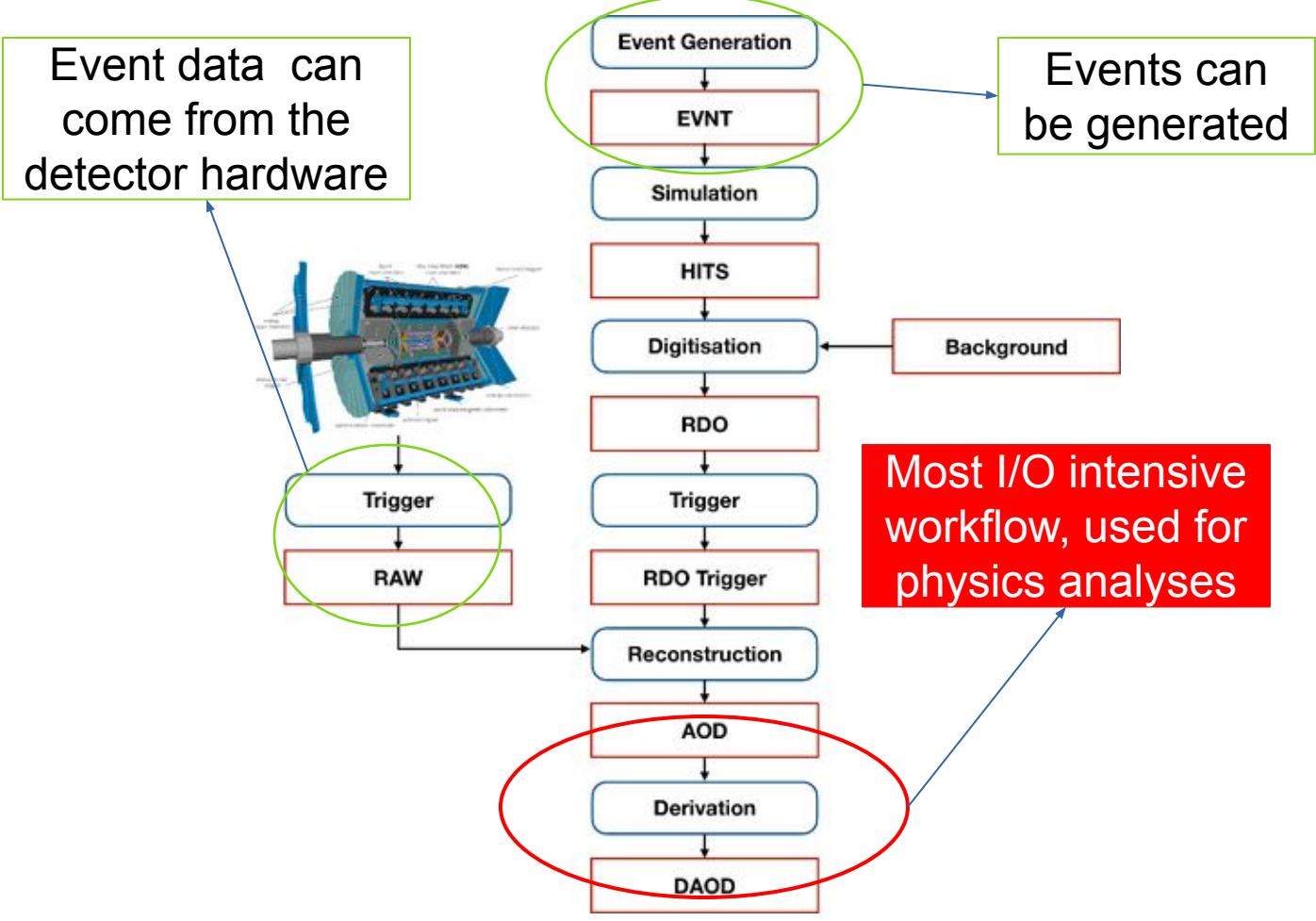


Evaluating I/O Behavior of ATLAS Derivations Across Container Runtimes on HPC Platforms

Wesley Kwiecinski (wkwie@uic.edu)
Michael E. Papka (papka@uic.edu)
University of Illinois Chicago

Introduction

The ATLAS experiment[1] is one of the particle physics experiments supported by the Large Hadron Collider (LHC)[2] at CERN. The increasing amount of data collected and processed has led CERN to using super computers outside of the Worldwide LHC Computing Grid.

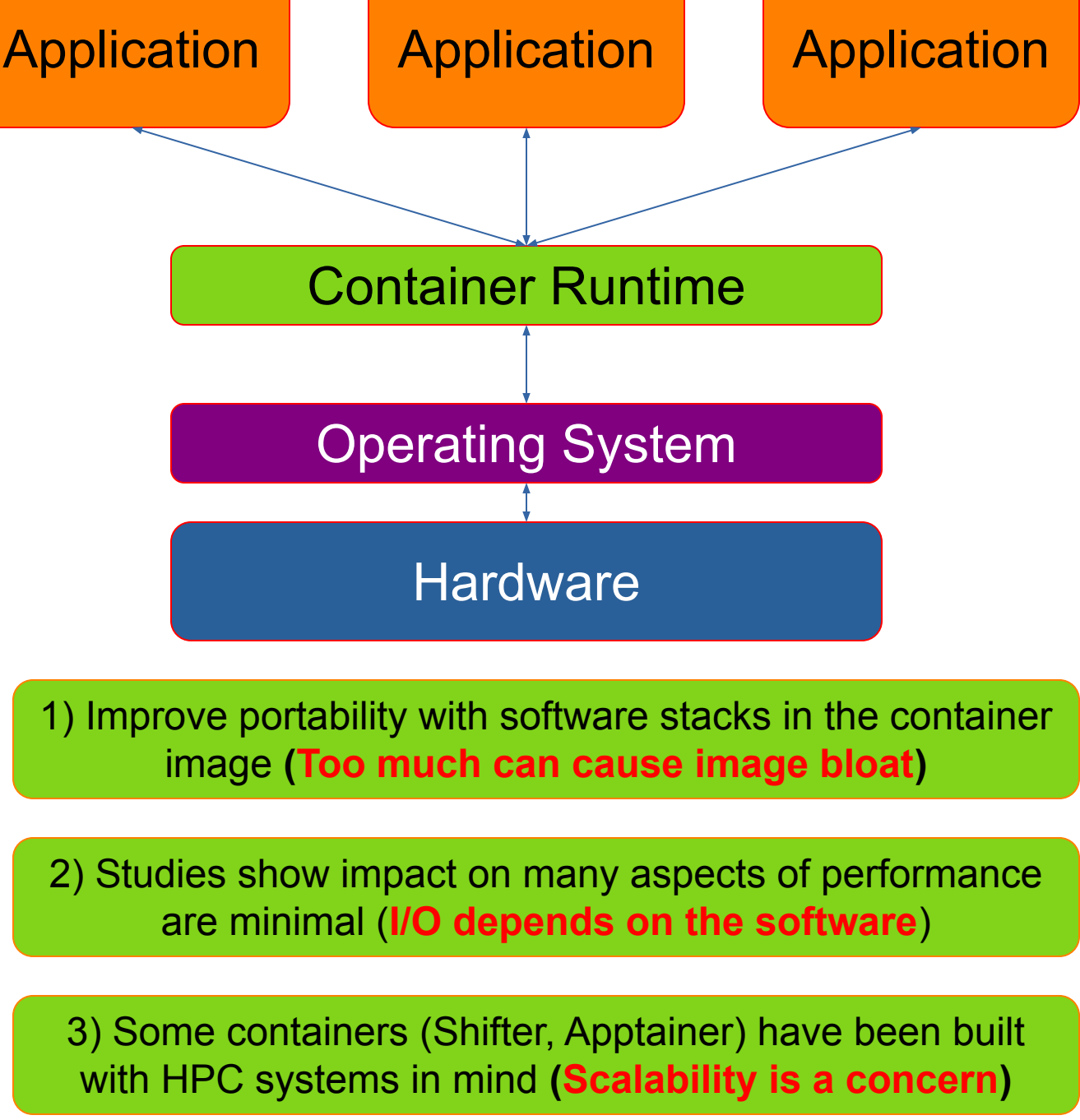


ATLAS data processing workflows are employed through Athena – the ATLAS software framework[3]. Athena has a complex software stack built up over more than a decade.

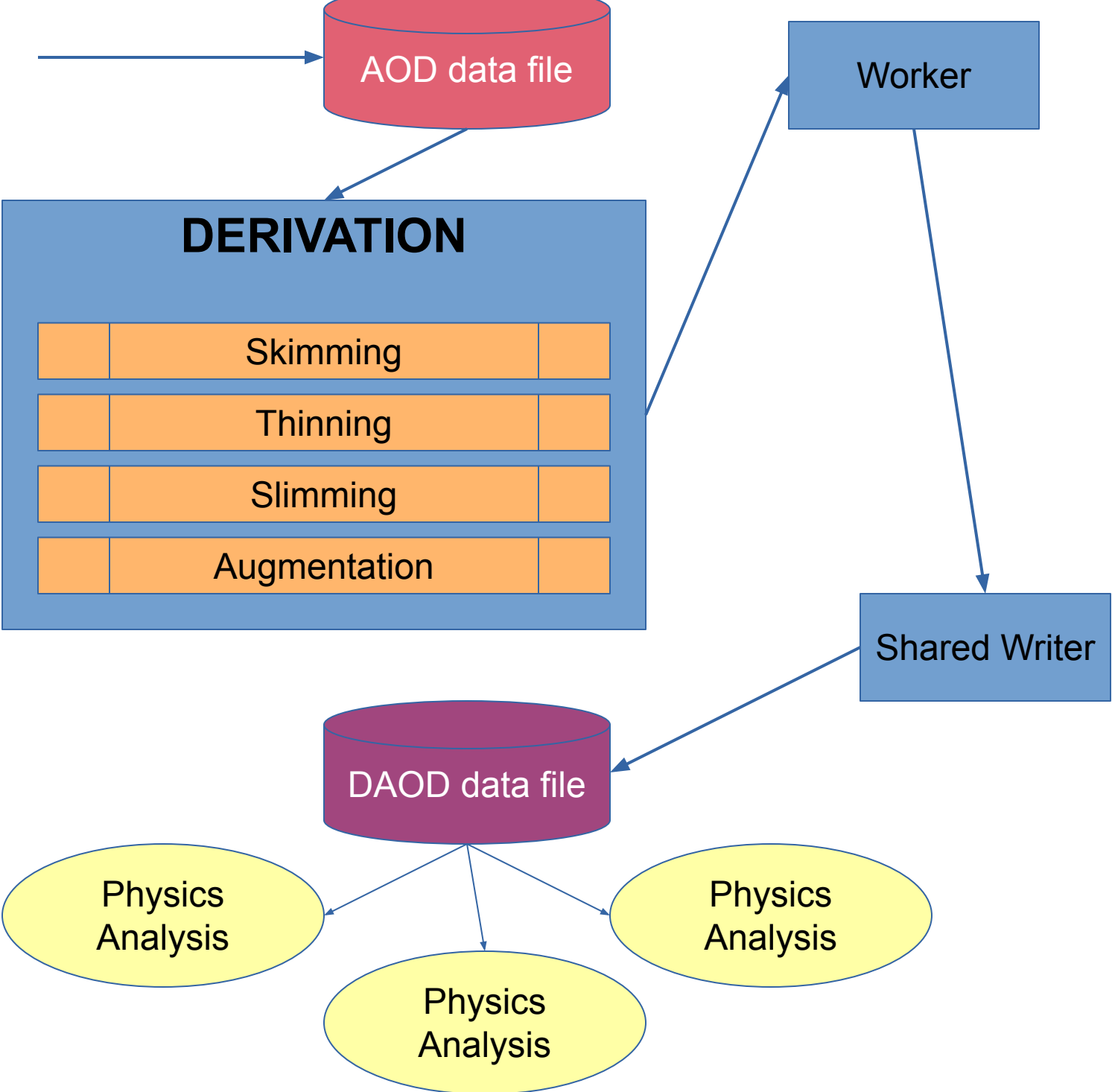
Considerations of Bringing ATLAS Workflows to HPCs

- 1) Distribution of complex software stacks to multiple complex hardware configurations
- 2) High I/O load of ATLAS workflows strains shared system resources which can affect other users
- 3) How to let ATLAS software make effective use of HPC hardware

Containers, or OS-level virtualizations, were adopted by CERN to improve the portability and reproducibility of their software.



Derivations are a high-intensity I/O process that are the start to most physics analyses. They perform a series of transformations on data from a Reconstruction job. Production derivations are not currently ran on HPCs.



Methodology

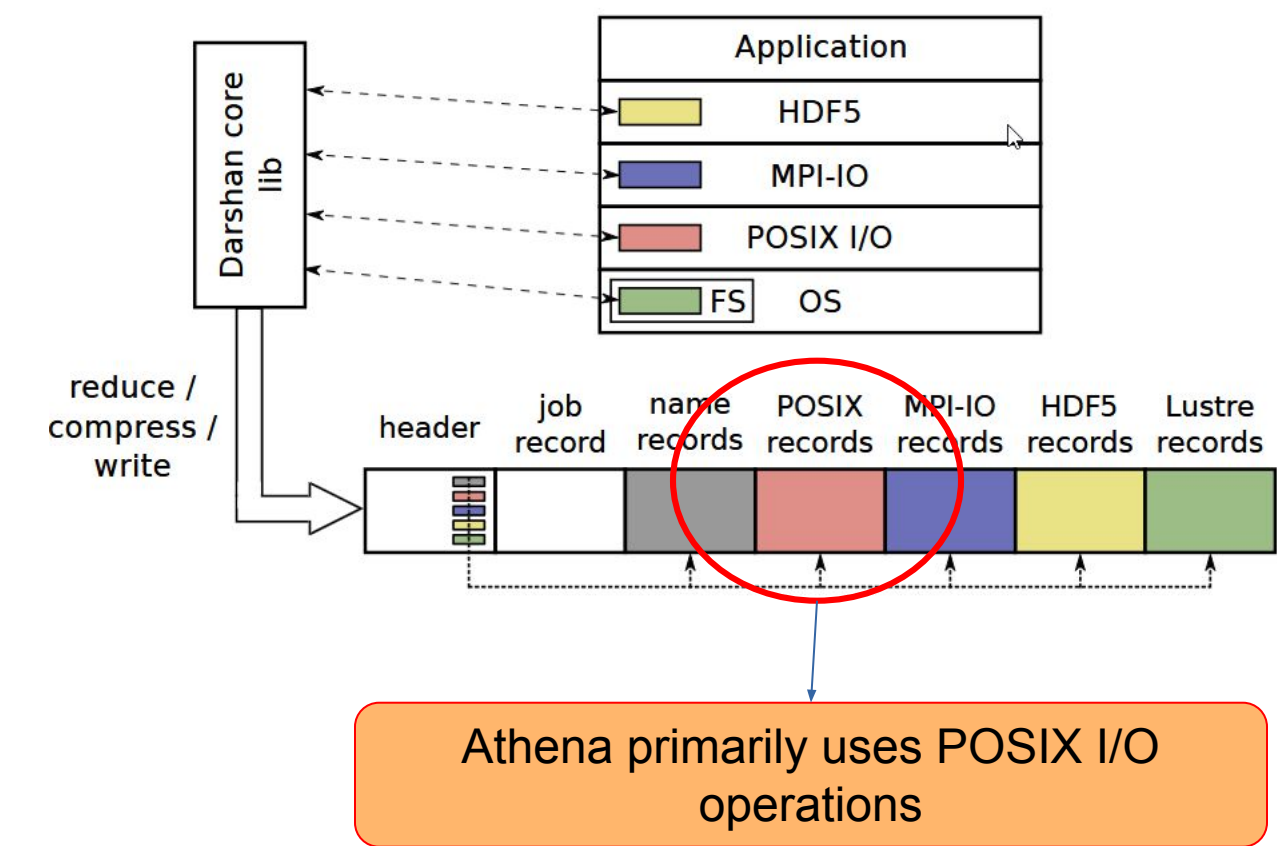
Experiments were performed on 2 different sets of hardware. Bare-metal experiments were performed on **bare-metal nodes via CERNs LXPLUS service**. Experiments using containers were ran on the **Perlmutter super computer at NERSC**.

System	CPU	RAM	NIC
aiatlasbm00X (3-5)	2x AMD EPYC 7302	258 GB	ICEC X550
Perlmutter	2x AMD EPYC 7763	512 GB DDR4	HPE Slingshot 11

Determine **weak** and **strong scaling** I/O performance for aspects of production derivations. Bare-metal experiments are ran using **Athena 25.0.51 on AlmaLinux 9.7**. Container experiments use a container image containing **AlmaLinux 9.7**. Experiments are ran with **up to 64 processes for bare-metal** and up to **128 processes for containers**. Containers used are all containers officially supported by ATLAS that can be used on HPCs: **Apptainer, Shifter, and Podman**

Experiment	Purpose	File System	Metrics
Weak Scaling Baseline	Production patterns w/ weak scaling	/tmp, CVMFS	Distribution of I/O operations
Weak Scaling Phases	Patterns in each derivation phase	/tmp, CVMFS	Changing patterns when adjusting params
Weak Scaling Shared FS	Impact on shared file system	EOS, AFS, Lustre scratch, CVMFS	File system latency
Strong Scaling Baseline	Production patterns w/ strong scaling	/tmp, CVMFS	Distribution of I/O operations
Strong Scaling Phases	Patterns in each derivation phase	/tmp, CVMFS	Chaging patterns when adjusting params
Strong Scaling Shared FS	Impact on shared file system	EOS, AFS, Lustre, scratch, CVMFS	File system latency

I/O performance data is collected via **Darshan**[4], an **application** I/O characterization tool developed by Argonne National Laboratory. To get the full picture of I/O, we trace a many files as possible used by the derivation workflow or surrounding container.



References

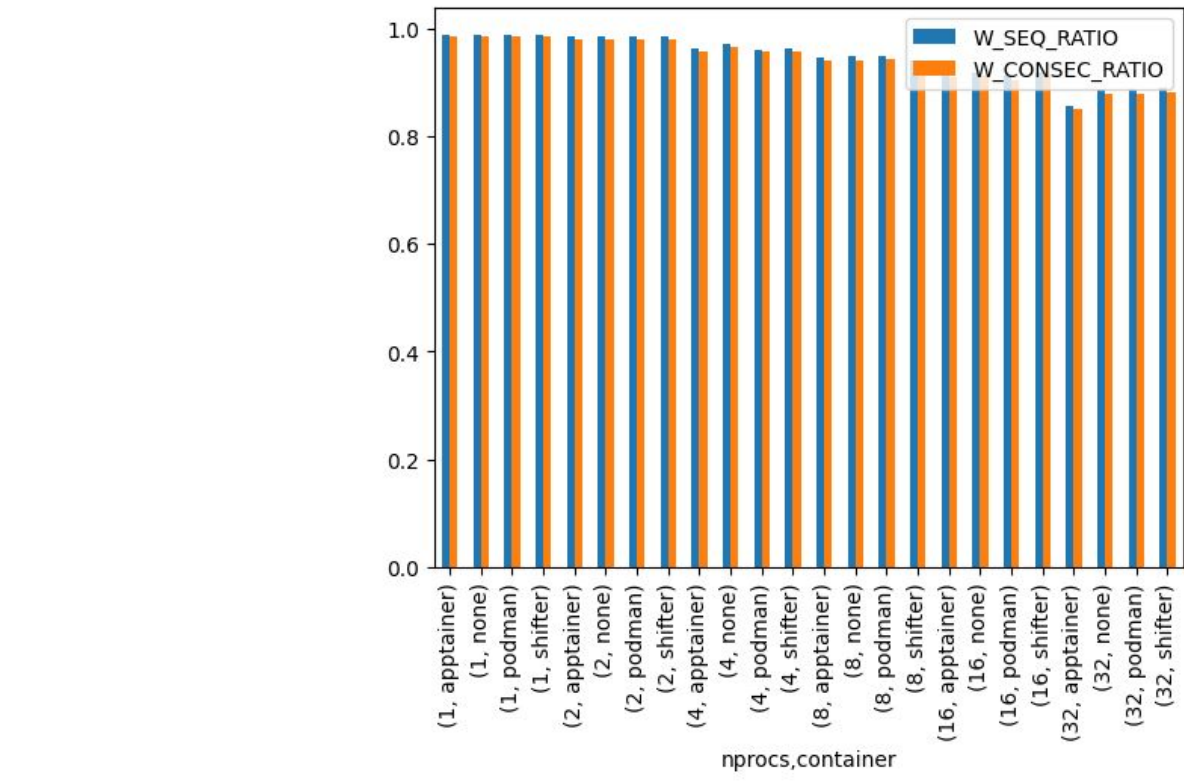
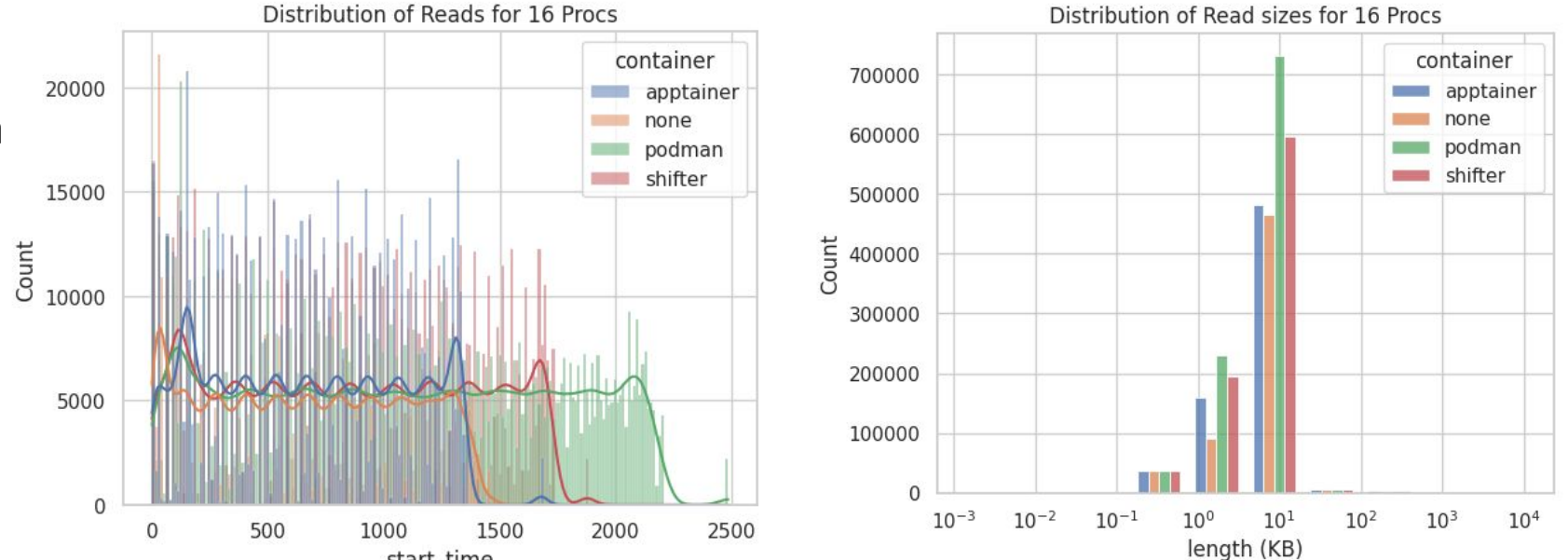
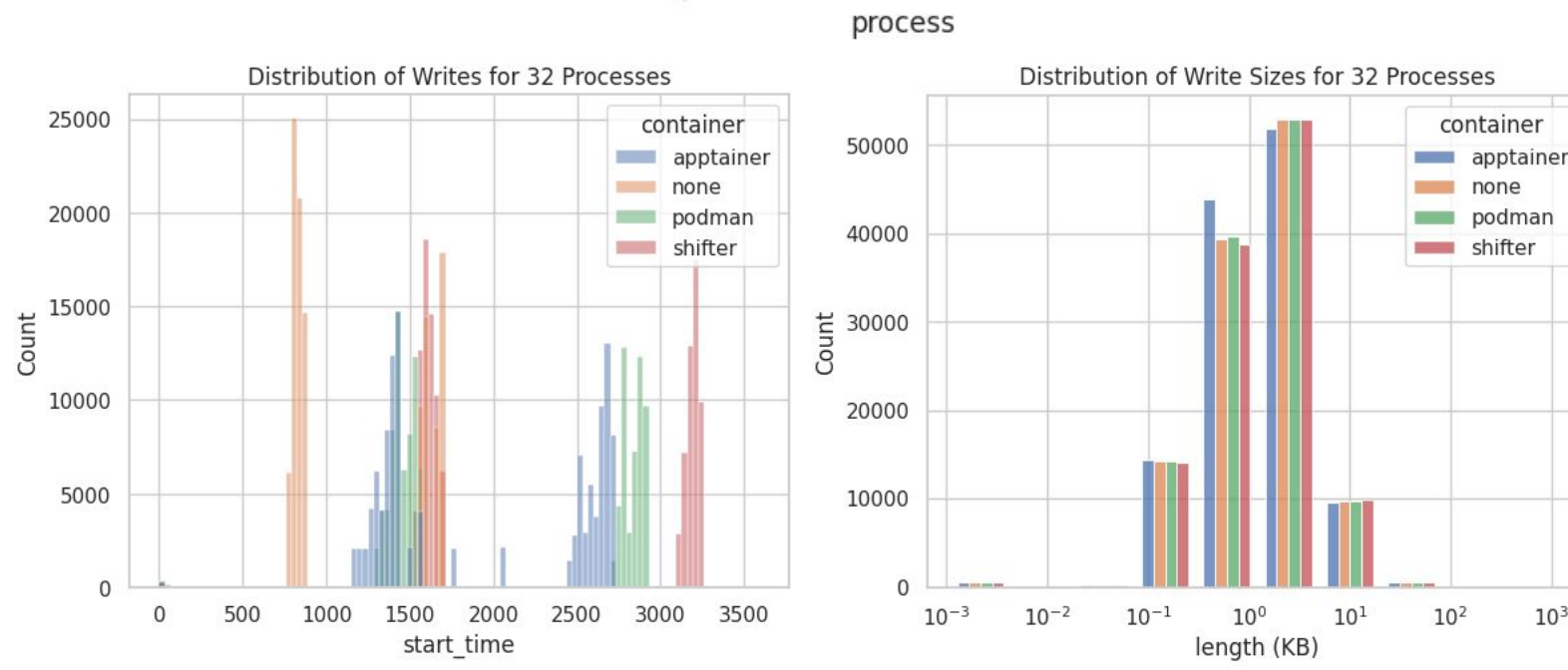
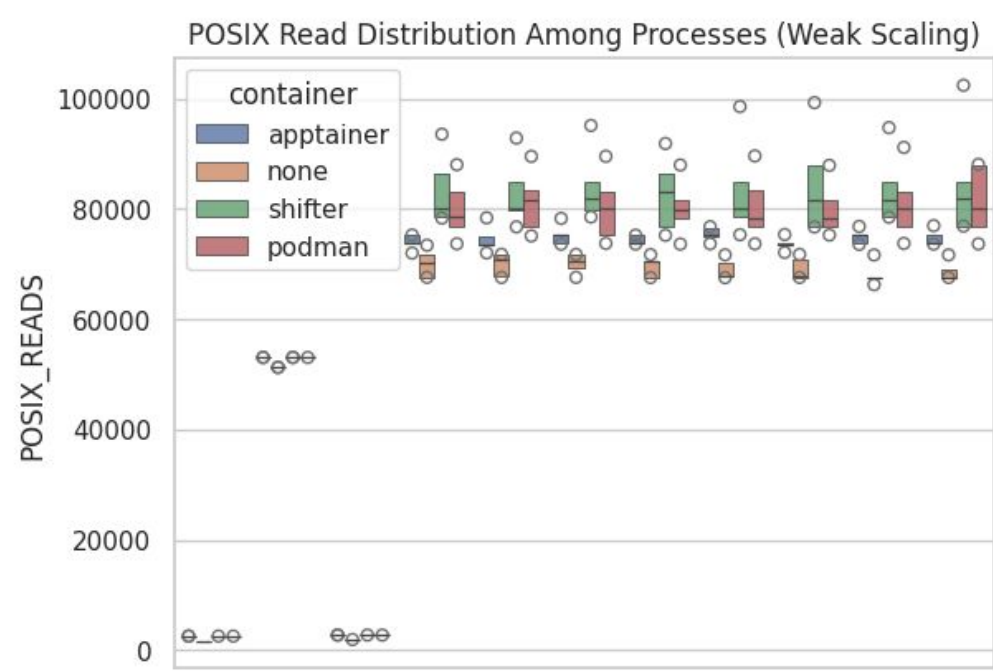
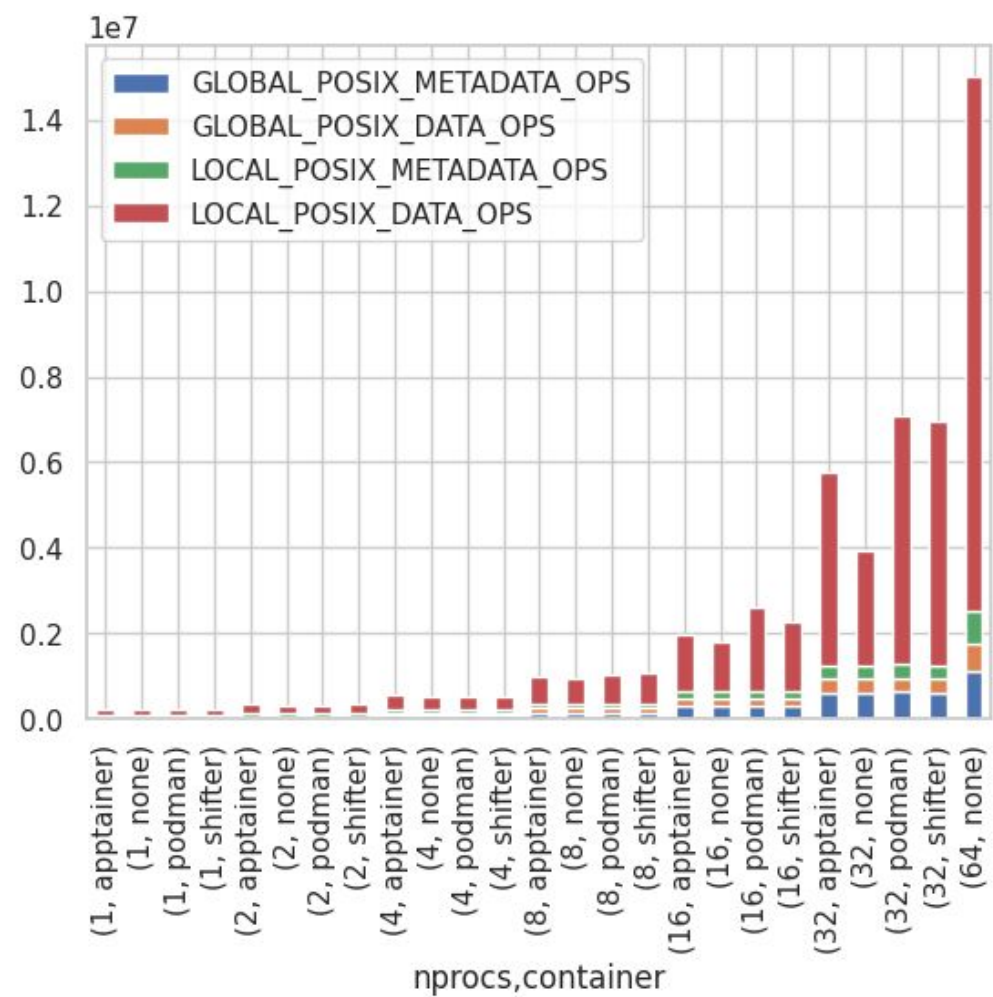
- [1] The ATLAS Collaboration: The atlas experiment at the cern large hadron collider, 2008
- [2] Bruning, O., Burkhardt, H., and Myers, S.: The large hadron collider. Progress in Particle and Nuclear Physics, 67(3):705–734, 2012.
- [3] The ATLAS Collaboration: Athena - atlas software documentation, Mar 2025
- [4] Snyder, Shane, et al. "Modular hpc i/o characterization with darshan." 2016 5th workshop on extreme-scale programming tools (ESPT). IEEE, 2016.

Acknowledgments



Experiment Results

Early results from baseline experiments give us insights into derivation I/O patterns among various container runtimes, and how well they scale.



Conclusion

- Preliminary results show:
- Majority of operations at scale a data transfer operations from 1 process
 - Apptainer is the most actively supported container and performs the closest to bare-metal
 - Consistent reads & clusters of writes
 - Scaling issues start to emerge past 16 processes

Future Work

- Remaining questions:
- What is the impact on file system latency?
 - What about container architecture impacts I/O patterns?
 - Can we easily adjust I/O patterns in a given container?